

1. HTML5 Form Controls & Input Types

HTML FORM

<!-- Inputs: Text, Number, Password, Date -->
<input type="text" id="uName" name="user" placeholder="Name">
<input type="number" id="valln" name="qty" min="0" step="any">
<input type="password" id="pwdln" name="pass">
<input type="date" id="dateIn" name="dt">
<!-- Dropdown & Textarea -->
<select id="catSel" name="cat">
 <option value="CSE">CSE</option>
 <option value="EEE">EEE</option>
</select>
<textarea id="txtArea" name="desc" rows="2"></textarea>
<!-- Radios & Checkboxes -->
<input type="radio" name="lvl" value="UG" checked> UG
<input type="radio" name="lvl" value="PG"> PG
<input type="checkbox" name="skill" value="JS"> JS
<input type="checkbox" name="skill" value="PHP"> PHP
<!-- Action Button & Output Containers -->
<button type="button" id="btnSubmit" onclick="runApp()">Submit</button>
<p id="output"></p>

2. DOM Extraction, Parsing & Controls

DOM CORE

// 1. Reading & Strict Conversions
let str = document.getElementById("uName").value;
let intN = parseInt(document.getElementById("valln").value, 10);
let fitN = parseFloat(document.getElementById("valln").value);
let num = Number(document.getElementById("valln").value);

// 2. Radios & Checkboxes
let radVal = document.querySelector('input[name="lvl"]:checked').value;
let checkedArr = Array.from(document.querySelectorAll('input[name="skill"]:checked')).map(cb => cb.value);

// 3. Writing Output & Styling
let out = document.getElementById("output");
out.innerHTML = `<b>Score:</b> \${fitN}`;
out.style.color = (fitN >= 50) ? "green" : "red";

// 4. State Disabling & Clearing
document.getElementById("valln").value = ""; // clear
document.getElementById("valln").disabled = true; // lock
document.getElementById("btnSubmit").disabled = true; // lock
btn

Trap: value is ALWAYS a string! "20" + 5 = "205". Always parse with parseFloat() or parseInt()!

3. Dynamic HTML Table Row Builder

DOM INJECT

// Append row to existing <tbody id="tblBody">
let tbody = document.getElementById("tblBody");
tbody.innerHTML += <tr>
 <td>\${str}</td>
 <td>\${fitN}</td>
</tr>;

// Create element natively
let li = document.createElement("li");
li.textContent = ` \${str} - \${fitN}`;
document.getElementById("myList").appendChild(li);

4. Math API & Universal Random Formula

MATH ENGINE

let rnd = Math.floor(Math.random() * (max - min + 1)) + min;

Range	Exact Code Snippet
500 .. 5000	Math.floor(Math.random() * 4501) + 500
200 .. 400	Math.floor(Math.random() * 201) + 200
50 .. 150	Math.floor(Math.random() * 101) + 50
1 .. 100	Math.floor(Math.random() * 100) + 1
1 .. 6 (Dice)	Math.floor(Math.random() * 6) + 1

Math.abs(x - 80); // absolute difference: |-5| = 5
Math.round(x * 100)/100; // round to 2 dec: 90.666 -> 90.67
x.toFixed(2); // STRING "90.67" (display only)
Math.min(...arr); // lowest in array
Math.max(...arr); // highest in array

5. Form Validation & Submit Guard

VALIDATION

document.getElementById("myForm").onsubmit = function(e) {
 let v = document.getElementById("valln").value.trim();
 if (v === "" || isNaN(v)) {
 e.preventDefault(); // stop submission!
 document.getElementById("output").textContent = "Enter valid input!";
 }
};

6. Universal JS 5-Stage App Pipeline

Q1 MASTER

<input type="text" id="fName" placeholder="Item/Subject">
<input type="number" id="fVal" placeholder="Score/Hours/Cal">
<button id="btn" onclick="processApp()">Process</button>
<p id="msgOut"></p>
</script>

// STAGE 0: State Variables OUTSIDE Function!
let runningTotal = 0, sessionCount = 0;
let itemHistory = [];
let secretTarget = Math.floor(Math.random() * 4501) + 500;

function processApp() {
 // STAGE 1: Extract & Validate
 let name = document.getElementById("fName").value;
 let val = parseFloat(document.getElementById("fVal").value);
 let out = document.getElementById("msgOut");

 // STAGE 2: Mutate State & History
 runningTotal += val;
 sessionCount++;
 itemHistory.push(name);

 // STAGE 3: Statistical Computation
 let avg = runningTotal / sessionCount;

 // STAGE 4: Categorical Evaluation Ladder (Highest First!)
 let feedback = "";
 if (runningTotal >= 100) feedback = "Goal Achieved!";
 else if (runningTotal >= 60) feedback = "Good Progress!";
 else feedback = "Keep Working!";

 // Guard: Warn if high attempts without goal
 if (sessionCount > 7 && runningTotal < 100) {
 feedback += " (High Attempts Warning!)";
 }

 // STAGE 5: DOM Feedback Output
 out.innerHTML = `Total: \${runningTotal} | Avg: \${avg.toFixed(2)} | + History: \${itemHistory.join(", ")} | Status: <b>\${feedback}</b>`;

 // STAGE 6: Termination & Disabling
 if (runningTotal >= 100 || sessionCount >= 10) {
 document.getElementById("fVal").disabled = true;
 document.getElementById("btn").disabled = true;
 }
</script>

7. Specialized JS Logic Engines

ALGORITHMS

// Engine A: Target Comparator & Difference
let diff = Math.abs(actual - target);
if (actual === target) msg = "Target met exactly (0 difference)";
else if (actual > target) msg = "Above target by \${diff}";
else msg = "Below target by \${diff}";

// Engine B: Secret Guess Machine (5 Attempts)
if (guess === secret) {
 feed = "Correct!"; input.disabled = true;
} else if (attempts >= 5) {
 feed = "Out of guesses!"; input.disabled = true;
} else if (guess > secret) { feed = "Too high!"; }
else { feed = "Too low!"; }

// Engine C: Rule Scoring & Regex Evaluator
let score = 0;
if (p.length >= 6) score += Math.floor((p.length - 6)/2) * 10;
if (/[A-Z]/.test(p)) score += 15; // Upper
if (/[a-z]/.test(p)) score += 15; // Lower
if (/0-9/.test(p)) score += 20; // Digit
if (/!@#\$%^&*/.test(p)) score += 25; // Symbol

// Engine D: Dual-Metric Formula (Health Risk)
let risk = (100 - spo2) * 2 + Math.abs(hr - 80) * 0.5;
let tier = (risk <= 10) ? "SAFE" : (risk <= 20 ? "WARNING" : "DANGER");

8. Financial, Discount & Tax Calculator

BILLING

let subtotal = price * qty;
let discountPct = 0;
if (subtotal >= 5000) discountPct = 0.20; // 20% off
else if (subtotal >= 2000) discountPct = 0.10; // 10% off
let discountAmt = subtotal * discountPct;
let taxAmt = (subtotal - discountAmt) * 0.05; // 5% VAT
let grandTotal = subtotal - discountAmt + taxAmt;

9. Dynamic Timer & Stopwatch Engine

TIMING

let sec = 0, timerId = null;
function startTimer() {
 if (!timerId) {
 timerId = setInterval(() => {
 sec++;
 document.getElementById("clock").textContent = `\${sec}s`
 }, 1000);
 }
}
function stopTimer() { clearInterval(timerId); timerId = null; }

10. JS Array, Object & Filter Toolkit

UTILITY

// Array Operations
let arr = [10, 20, 30];
arr.push(40); // append element
arr.pop(); // remove last element
arr.join(","); // "10, 20, 30"
arr.slice(0, 2); // copy sub-array without mutating
arr.sort((a, b) => a - b); // ascending numeric sort
arr.sort((a, b) => b - a); // descending numeric sort

// String Splitting & Cleanup
"English-3".split("-"); // ["English", "3"] (subject + hr)
str.trim(); // remove whitespace
str.toLowerCase(); // case-insensitive comparison
str.includes("Beach"); // substring check

// Search in Array of Objects
let filtered = items.filter(item => item.name.toLowerCase().includes(query.toLowerCase()) && item.price <= maxPrice);

11. Weighted GPA / Grade Calculator

GRADING

function getGP(marks) {
 if (marks >= 90) return 4.00;
 if (marks >= 80) return 3.50;
 if (marks >= 70) return 3.00;
 if (marks >= 60) return 2.50;
 return 0.00;
}
let totalPoints = 0, totalCredits = 0;
totalPoints += getGP(courseMarks) * courseCredits;
totalCredits += courseCredits;
let gpa = totalPoints / totalCredits;

12. PHP One-File Form Processor

PHP CORE

<form method="post" action="">
 Input A: <input type="number" name="valA" required>
 Input B: <input type="number" name="valB" required>
 <input type="submit" name="btnSubmit" value="Calculate">
</form>
<?php
if (isset(\$_POST["btnSubmit"])) {
 \$a = floatval(\$_POST["valA"]);
 \$b = floatval(\$_POST["valB"]);
 \$res = \$a * \$b;
 echo "<p>Result " . \$res . "</p>";
}
?>

PHP Rules: Variables always start with \$. String concatenation is dot (.), NOT +! Button must have name="btnSubmit".

13. PHP Functions & Sessions

PHP API

// Math Functions
ceil(30 / 8); // 4 (round UP to whole item)
floor(30 / 8); // 3 (round DOWN)
round(3.754, 2); // 3.75
min(\$a, \$b, \$c); // lowest (best 2 of 3 marks)
number_format(8400); // "8,400" (comma formatted)

// Sessions & Sanitization
session_start();
\$_SESSION["user"] = \$name; // store session
if (isset(\$_SESSION["user"])) { ... }
session_destroy(); // logout
htmlspecialchars(\$str); // XSS prevention

14. PHP Array Functions & Loops

ARRAYS

\$scores = ["Ali">85, "Bob">92, "Sara">78];
asort(\$scores); // sort by value ASC (asort for DESC)
foreach (\$scores as \$student => \$mark) {
 echo "{\$student}: {\$mark}";
}
\$avgScore = array_sum(\$scores) / count(\$scores);

15. The Universal ceil() Allocator Engine

Q2 CORE

"Cannot order/book partial items" → Always ceil():

```
// 1. Universal 5-Line Math Formula
$needed = $demandCount * $ratePerUnit;
$items = ceil($needed / $capacityPerItem); // Round UP
$supplied = $items * $capacityPerItem; // Total capacity
$unused = $supplied - $needed; //
Leftover/empty
$wasted = $unused * $costPerSingleUnit; // Money
wasted

// 2. Efficiency Ratio / Category
$ratio = $unused / $supplied;
if ($ratio <= 0.10) $status = "Efficient";
elseif ($ratio <= 0.25) $status = "Acceptable";
else $status = "Wasteful";

Distractor Rule: Fixed venue/screen rental (15,000 / 25,000 BDT)
is NEVER in the wasted formula! Wasted money is strictly unused * unitPrice!
```

16. Progress Classifier & Best-of-3

PHP LOGIC

```
// Pattern A: Best 2 of 3 CT Marks (Drop lowest)
$ctAvg = ($ct1 + $ct2 + $ct3 - min($ct1, $ct2, $ct3)) / 2;
$total = $ctAvg + $midterm + $final;
$status = ($total > 54) ? "Passed" : "Failed";

// Pattern B: Progress Classifier & Target Delta
$total = $rate * $count; // items*days OR credits*courses
if ($total >= 500) $tier = "Excellent"; // 261: 120
elseif ($total >= 300) $tier = "Good"; // 261: 90
elseif ($total >= 150) $tier = "Average"; // 261: 60
else $tier = "Poor";

// Target Comparison Delta
if ($total == $target) {
    $msg = "Target met exactly (0 difference)";
} elseif ($total > $target) {
    $msg = "Above target by " . ($total - $target);
} else {
    $msg = "Below target by " . ($target - $total);
}
```

17. PHP OOP Class Architecture

OOP SPEC

```
class Product {
    public $title;
    public $price;
    public $qty;

    // Constructor (TWO underscores: _construct)
    public function __construct($t, $p, $q) {
        $this->title = $t;
        $this->price = $p;
        $this->qty = $q;
    }

    public function calculateTotal() {
        return $this->price * $this->qty;
    }

    public function printDetails() {
        echo "Title: " . $this->title . "<br>";
        echo "Total: " . $this->calculateTotal() . "<br>";
    }
}

// Object Instantiation & Calling
$item = new Product("Web Book", 500, 3);
$item->printDetails();

OOP Rules: $this->property (NO $ after ->). Method calls use
$obj->method().
```

18. PHP Date & Overdue Days Engine

DATES

```
// Day difference between two dates
$due = strtotime("2026-09-10");
$curr = strtotime("2026-09-22");
$daysOverdue = ($curr - $due) / 86400; // 86400s per day
if ($daysOverdue > 0) {
    $penalty = $daysOverdue * 2.00; // 2 BDT per day
}
```

19. PHP Regex Validation & Cookies

SECURITY

```
// Email & Code Regex
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) { $err = "Invalid email"; }
if (!preg_match('/^[A-Z][0-9]{3}$/i', $courseCode)) { /* Valid format */ }

// Cookies (Name, Value, Expiry in seconds)
setcookie("pref_theme", "dark", time() + 86400 * 7);
$theme = $_COOKIE["pref_theme"] ?? "light";
```

20. SQL DDL & Data Types

SCHEMA SETUP

```
CREATE DATABASE IF NOT EXISTS uiu_db;
USE uiu_db;

CREATE TABLE records (
    id INT PRIMARY KEY AUTO INCREMENT,
    title VARCHAR(60) NOT NULL,
    category VARCHAR(30),
    stock INT DEFAULT 0,
    fee DECIMAL(10,2), -- Money: 0.00
    rating FLOAT, -- 4.5
    reg_date DATE, -- '2026-09-23'
    status VARCHAR(20)
);

INSERT INTO records (title, category, stock, fee, status)
VALUES
('Web Dev', 'CSE', 10, 45.00, 'Active'),
('Database', 'CSE', 5, 30.00, 'Overdue');

Quote Escape: Double single quotes to insert apostrophe: 'Cox's Bazar'.
```

21. Deep SQL Query Patterns

WEAK POINT FOCUS

Pattern 1 — Selective Filters (OR, LIKE, BETWEEN, IN):

```
SELECT title, category, rating FROM records
WHERE rating > 4.5 OR title LIKE '%Beach%'
ORDER BY rating DESC;

SELECT * FROM records
WHERE rating BETWEEN 4.0 AND 4.5; -- Inclusive!

SELECT * FROM records WHERE category IN ('CSE', 'EEE');
```

Pattern 2 — Group Aggregations with HAVING:

```
-- HAVING filters aggregated groups (WHERE cannot!)
SELECT category, COUNT(*) AS total_items, AVG(rating) AS avg_rate
FROM records
GROUP BY category
HAVING COUNT(*) > 1 AND AVG(rating) > 4.4
ORDER BY avg_rate DESC;
```

Golden Rule: Every non-aggregated selected column MUST be in GROUP BY! Use HAVING for aggregate conditions, never WHERE!

Pattern 3 — Guarded Conditional UPDATE:

```
-- Arithmetic update with ceiling guard (prevent overshoot)
UPDATE records
SET fee = fee * 1.10
WHERE fee > 20.00 AND fee * 1.10 <= 50.00;

-- Conditional grade swap with exclusion
UPDATE records
SET status = 'Grace Period', fee = 0
WHERE status = 'Overdue' AND days_overdue < 7;
```

22. Subqueries, Computed Totals & Joins

ADVANCED SQL

```
-- 1. Grand Net Worth (Multiplication inside SUM)
SELECT SUM(fee * stock) AS grand_total FROM records;

-- 2. Subquery: Select items above overall average
SELECT title, fee FROM records
WHERE fee > (SELECT AVG(fee) FROM records);

-- 3. Correlated Subquery CASE (Top Seller in its OWN category)
SELECT title, fee,
CASE
    WHEN fee > (SELECT AVG(r2.fee) FROM records r2
    WHERE r2.category = r1.category)
    THEN 'Top Seller' ELSE 'Regular Seller'
END AS seller_tier
FROM records r1;

-- 4. INNER & LEFT JOIN across two tables
SELECT e.name, d.dept_name, COUNT(p.id) AS proj_count
FROM emp e
LEFT JOIN dept d ON e.dept_id = d.id
LEFT JOIN project p ON e.id = p.emp_id
GROUP BY e.id;
```

23. SQL Built-ins & Guarded DELETE

SQL EXTRA

```
-- String & Date built-in functions
SELECT CONCAT(first_name, ' ', last_name) AS full_name,
UPPER(category) AS cat_upper,
DATEDIFF(NOW(), reg_date) AS days_registered
FROM records;

-- Guarded DELETE (ALWAYS specify precise WHERE!)
DELETE FROM records WHERE status = 'Inactive' AND stock < 0;
```

24. Master PHP ↔ MySQL Bridge Skeleton

WRITE IN Q3

"Write full PHP-MySQL code, not just SQL" → Write this entire file!

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "uiu_db";

// 1. Establish Connection
$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// 2. SELECT Query Execution & Fetch Loop
$sql1 = "SELECT category, COUNT(*) AS total_items, AVG(fee) AS avg_fee
FROM records
GROUP BY category
HAVING COUNT(*) > 1
ORDER BY avg_fee DESC";

$result1 = $conn->query($sql1);

if ($result1 && $result1->num_rows > 0) {
    while ($row = $result1->fetch_assoc()) {
        echo "Category: " . $row["category"] .
        " | Total: " . $row["total_items"] .
        " | Avg Fee: " . $row["avg_fee"] . "<br>";
    }
} else {
    echo "0 results found<br>";
}

// 3. UPDATE Query Execution with Feedback
$sql2 = "UPDATE records
SET fee = fee * 1.10
WHERE fee > 20.00 AND fee * 1.10 <= 50.00";

if ($conn->query($sql2) === TRUE) {
    echo "Record updated successfully (" . $conn->affected_rows .
    " rows)<br>";
} else {
    echo "Error updating: " . $conn->error . "<br>";
}

// 4. Close Connection
$conn->close();
?>
```

Column Alias Law: \$row["total_items"] works ONLY because of AS total_items in SQL! Always alias aggregated columns.

25. HTML Table Rendering from Database

HTML OUTPUT

```
echo "<table border='1' cellpadding='4' cellspacing='0'>
<tr>
<th>ID</th><th>Title</th><th>Fee</th>
</tr>";

while ($row = $result->fetch_assoc()) {
    echo "<tr>
<td> " . $row["id"] . " </td>
<td> " . $row["title"] . " </td>
<td> " . $row["fee"] . " </td>
</tr>";
}
echo "</table>";
```

26. Dynamic Search Form & Prepared SQL

SECURITY

```
// HTML Search Form → PHP Prepared Query
$search = "%" . $_GET["keyword"] . "%";
$stmt = $conn->prepare("SELECT * FROM records WHERE title LIKE ? AND fee > ?");
$stmt->bind_param("sd", $search, $minFee); // s=str, d=double, i=int
$stmt->execute();
$res = $stmt->get_result();
while ($r = $res->fetch_assoc()) { echo $r["title"]; }
$stmt->close();
```

27. MySQL Transactions & Procedural

FULL BRIDGE

```
// Procedural mysqli alternative
$c = mysqli_connect("localhost", "root", "", "uiu_db");
$res = mysqli_query($c, "SELECT * FROM records");
while ($row = mysqli_fetch_assoc($res)) { echo $row["title"]; }
mysqli_close($c);

// Safe Transaction Block
$conn->begin_transaction();
try {
    $conn->query("UPDATE accounts SET bal = bal - 100 WHERE id = 1");
    $conn->query("UPDATE accounts SET bal = bal + 100 WHERE id = 2");
    $conn->commit();
} catch (Exception $e) { $conn->rollback(); }
```